

Vets4Warriors staffing forecast

How a call log becomes a staffing number

A plain-language walk through the whole pipeline: the data, the forecast model, the queueing math, the screen, and what is still unproven.

Student MVP built for the UTDesign EPICS project with Vets4Warriors. All figures in this document come from the repository files and the project database as of 2026-09-13. The database holds **seeded synthetic data**, not the client's real call log. Every accuracy number here describes performance on that seed data only.

1. One-page summary

What it does. The tool reads a table of past inbound calls to the Vets4Warriors peer-support line, learns the shape of demand by hour of day, day of week, month, holiday, and long-term trend, and predicts how many calls will arrive in each future hour. It then converts each hour's predicted calls into the number of people who must be on the phones to answer 80% of callers within 30 seconds, adjusts for the fact that a scheduled person is only free for calls part of the time, and rolls the hours up into three shifts: Day (08:00 to 16:30), Evening (16:00 to 00:30), and Overnight (00:00 to 08:30). The output is a table: for each date and each shift, how many people to schedule.

Who uses it. Supervisors and managers building a monthly schedule. Two sliders let them change the assumptions live: how much of a shift a person is actually available for inbound calls (50% to 85%), and how fast calls should be answered (80% to 95% within 30 seconds).

How good it is, on the seed data. The forecast was tested by pretending it was the first day of each month from January 2024 through December 2025, training only on earlier data, and predicting the whole month ahead. Across those 24 months the predicted daily total was off by 9.28 calls on a typical day, against an average of 51.15 calls per day. A simple rule of thumb (same weekday and hour, median of the last 8 weeks) was off by 10.34. In staffing terms: read any shift number as plus or minus one person.

What it is not.

- It has never seen the client's real data. The 93,407 calls in the database were generated by a script to look plausible. The accuracy numbers above say the pipeline works; they say nothing about how well it will work on real calls.
- It does not build a roster. It says how many people, not which people.
- It only covers inbound calls. Outbound follow-up work is not forecast and is not subtracted from capacity.
- It assumes callers never hang up while waiting. They do (8.06% of seed calls were abandoned). The math that handles that, Erlang A, is a known next step.
- It flags but does not solve the client's cap of 7 people per shift. If the math says 8, the screen shows 8 in red.

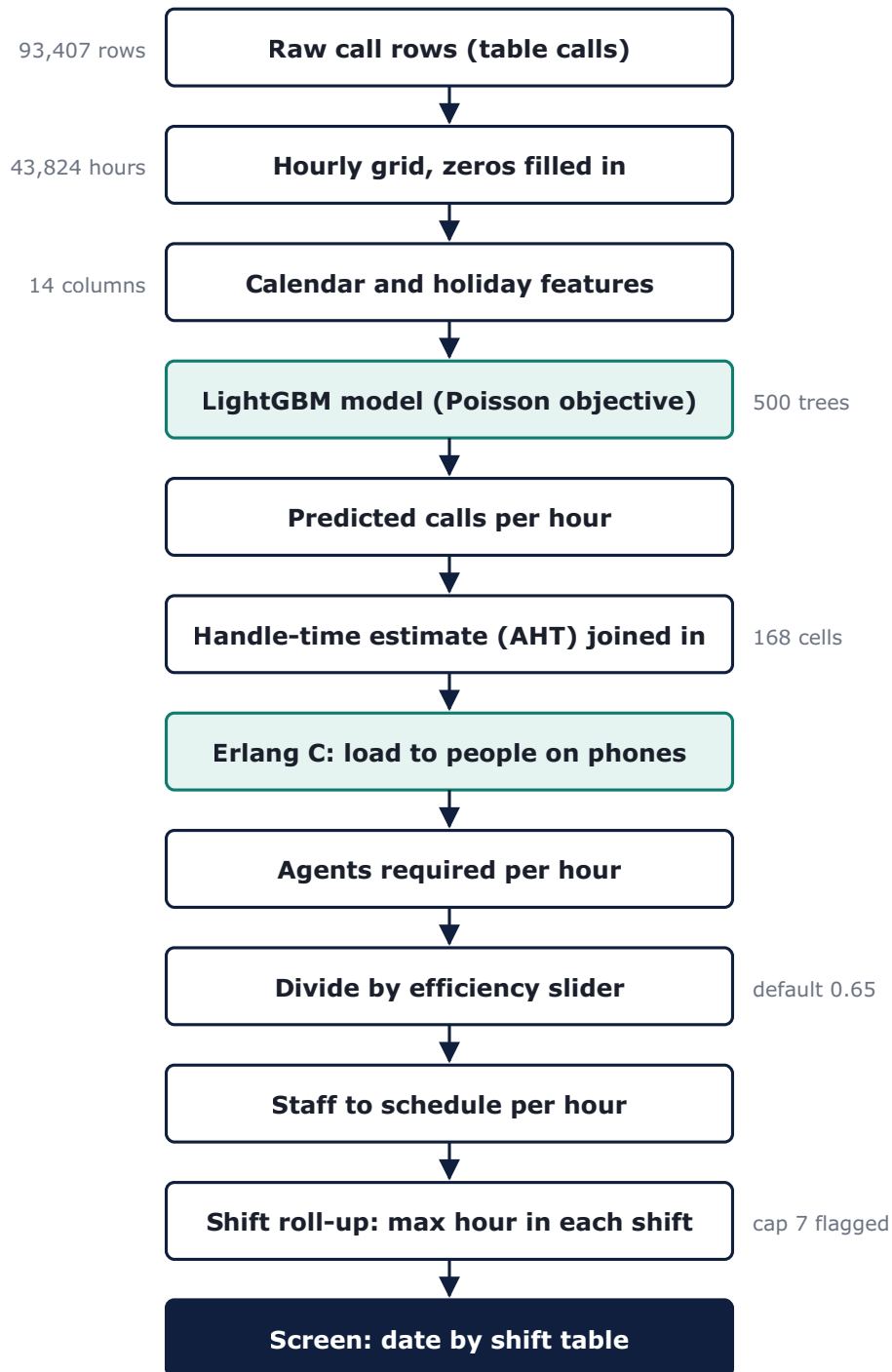
The one idea to keep

Every staffing number is a chain of four steps: predicted calls per hour, times minutes per call, gives busy-person-hours (the load); Erlang C turns the load into people needed on the phones; dividing by the efficiency slider turns that into people to schedule; and each shift takes the largest hourly answer inside it. Change any link and the number moves in a way you can trace by hand. Section 6 does exactly that.

Contents

1. One-page summary	2
2. The pipeline in one picture	4
3. The data	6
4. Every design decision, with its trade-off	9
5. How the model was trained	14
6. From predicted calls to a staffing number	17
7. What each control on the screen actually does	22
8. Known limitations and what's next	24
9. Glossary	26

2. The pipeline in one picture



Teal boxes are the two places where math does real work: the forecast model and the queueing formula. Everything else is bookkeeping.

What changes at each arrow

Raw rows to hourly grid. Each call is one row with a timestamp. The database view `calls_hourly` counts the rows in each clock hour. That view only has a row for hours with at least one call (35,872 of them), so the training script adds the missing 7,952 hours with a count of zero. Now there is one row for every hour from 2021-01-01 00:00 to 2025-12-31 23:00.

Hourly grid to features. For each hour the code writes down things you can know in advance: weekday, hour, month, week of year, how many days since the data began, whether it is a weekend, whether it is a holiday, how many days to the nearest holiday, and two holiday strength numbers. Nothing about how many calls arrived is used as a feature.

Features to LightGBM. LightGBM is a program that builds hundreds of small decision trees, each one correcting the errors of the previous ones. It is trained to predict the call count from the features. The "Poisson objective" tells it the target is a count of events, which changes how it scores its own mistakes (Section 4.4).

LightGBM to predicted calls. For any future hour, build the same features and ask the model. It answers with an expected number of calls, a decimal like 5.05, not a whole number, because it is an average over many possible outcomes.

Predicted calls plus AHT. Each hour of the week gets a typical call length in seconds, computed from past answered calls with recent calls counting more. This is looked up by (weekday, hour).

Erlang C. Predicted calls times seconds per call gives how many person-hours of talking will happen in that hour. Erlang C answers: with N people on the phones, what share of callers is answered within 30 seconds? The code raises N until the answer clears the target.

Agents to staff. A scheduled person is not free for inbound calls all shift (breaks, lunch, follow-up calls, paperwork). The efficiency slider says what share they are free. Divide agents by that share and round up.

Staff per hour to shifts. A shift takes the largest hourly staff number inside its window. If the requirement is above 7 the row is marked, never trimmed.

3. The data

The only data source is one Postgres database, reached through the environment variable `V4W_DATABASE_URL`. It has two tables and one view, defined in `db/schema.sql`. The rows in it were produced by a generator script and loaded once. The generator was written to mimic what the client described in meetings: a night-heavy line, Mondays busiest, holidays busier, calls getting longer at night, and volume growing year over year. It is scaffolding so the pipeline could be built and tested before real data arrives.

Object	Rows	Range
<code>calls</code>	93,407	2021-01-01 00:54:55 to 2025-12-31 23:55:58
<code>holidays</code>	91	2021-01-01 to 2027-12-31 (13 dates per year; 2026 and 2027 were added on 2026-09-13)
<code>calls_hourly</code> (view)	35,872	only hours that had at least one call

What the seed data looks like, measured directly:

Measure	Value
Answered calls	85,879
Abandoned calls	7,528 (8.06%)
Answered within 30 seconds, as a share of answered	84.13%
Talk time on answered calls: mean / median / maximum	703 s / 571 s / 7,200 s
Mean queue time on answered calls	27.3 s
Median wait before abandoning	69 s
Calls per day, averaged over all days	51.15
Calls per year 2021 / 2022 / 2023 / 2024 / 2025	16,035 / 16,454 / 18,383 / 20,937 / 21,598
Busiest hour of day (average calls in that hour)	21:00, 3.55
Quietest hour of day	05:00, 1.43
Most calls ever in one hour	18

3.1 The `calls` table, column by column

Six columns came from the client's description of their export. Three more are computed by the database itself.

Column	What it is	Used where
<code>call_start</code>	When the call arrived, local time, to the second.	Bucketed to the hour in the view; the hour and weekday drive every feature.
<code>inqueue_sec</code>	Seconds from arrival until an agent answered, including ringing. The client's service-level clock.	Only to compute <code>answered_30s</code> . Not used by the model or the staffing math.
<code>routing_sec</code>	The ringing part of the wait, at the end. Zero for abandoned calls.	Not used.
<code>agent_sec</code>	Seconds an agent spent talking. Zero for abandoned calls.	The handle-time estimate (AHT). Also summed into <code>talk_sec</code> in the view.
<code>abandoned</code>	True if the caller hung up before an agent answered.	Filters answered calls for AHT. Counted in the view.
<code>abandon_sec</code>	How long an abandoning caller waited before giving up. Zero for answered calls.	Not used. This is the column an Erlang A upgrade would need (Section 4.8).
<code>call_hour</code> (computed)	Hour of <code>call_start</code> , 0 to 23.	AHT lookup key.
<code>dow</code> (computed)	Weekday, 1 = Monday to 7 = Sunday.	AHT lookup key.
<code>answered_30s</code> (computed)	True if answered and <code>inqueue_sec</code> is 30 or less. The service-level rule, defined once in the schema so nobody rewrites it slightly differently.	Counted in the view. Not used downstream yet.

The database also enforces the client's own rules: an abandoned call has zero talk time, zero routing time, and its abandon time equals its queue time; an answered call has zero abandon time and routing time no larger than queue time. Rows that break these rules cannot be inserted. There is no call ID in the export, so the six client columns together act as the row's identity, which makes re-loading the same file a no-op.

3.2 The holidays table

Column	What it is	Used where
holiday_date	The date. Primary key.	Sets is_holiday and days_to_nearest_holiday for every hour.
holiday_name	Label, for example "Veterans Day".	Display only.
volume_multiplier	How much busier the whole day is expected to be, for example 1.550 for Independence Day.	Feature holiday_volume_multiplier (0 on non-holidays).
evening_multiplier	An extra evening lift, for example 1.850 for Independence Day.	Feature holiday_evening_multiplier. Applied to all 24 hours, not just evenings; see Section 8.

The 13 dates each year are: New Year's Day, Valentine's Day, Memorial Day, Juneteenth, Independence Day, July 5 (spillover), Labor Day, September 11, Veterans Day, Thanksgiving, Christmas Eve, Christmas Day, New Year's Eve. The multipliers are the same every year. They came with the seed data; where they came from is not recorded.

3.3 The calls_hourly view

A view is a saved query that looks like a table but is recomputed from calls every time it is read, so it can never be stale. This one groups calls by clock hour and returns, per hour: hour_start, dow, call_hour, offered (calls that arrived), answered, abandoned, answered_30s, talk_sec (total talk seconds), and offered_load_erlangs (talk seconds divided by 3,600). The model uses only hour_start and offered. The other columns exist for the dashboard's history views, which are not built yet.

Why hours and not individual calls

A schedule is written in hours, not in calls. Nobody staffs for "the 14:07 call". Counting calls per hour turns 93,407 unpredictable events into 43,824 numbers with a clear daily and weekly rhythm, which is what a forecast model can learn. It also throws away the exact minute of each call, which is fine, because Erlang C only needs the rate.

4. Every design decision, with its trade-off

4.1 Hourly buckets instead of per-call modeling

Chose: predict the count of calls in each hour. **Alternatives:** predict each call's arrival time; use 15- or 30-minute buckets. **Why:** supervisors schedule by shift, the client's service level is defined per call but managed per hour, and hourly counts on this line are already small (mean 2.13 per hour). Finer buckets would be mostly zeros and ones, which no model can forecast well.

What it costs

Anything shorter than an hour is invisible. A surge that lasts 20 minutes is smeared across the hour and staffing for it is under-called. The 30-minute shift overlaps (08:00 to 08:30 and so on) cannot be modeled exactly, so the boundary hour is counted toward both shifts for the staffing peak.

4.2 Filling the full hourly grid with zeros

Chose: every hour in the range gets a row; hours with no calls get 0. **Alternative:** train only on the 35,872 hours that had calls, as the view returns them. **Why:** two problems with the alternative.

Leakage. The backtest rule is "predict a whole month with no access to that month's data". If the test rows are only the hours that turned out to have calls, then the choice of which hours to predict already uses knowledge of the test month. The predicted daily total would then skip the quiet hours, which is cheating without meaning to.

Overnight blindness. A model that never sees a zero will not predict values near zero. Overnight hours are where zeros live (18.1% of all hours are zero, almost all at night). Without them the model would overstate night demand, and staffing would be wrong exactly where the client said calls are rarest and longest.

What it costs

An absent hour is treated as "zero calls". If the real export ever has gaps for a different reason (a system outage, a missing day in the file), those gaps become false zeros and pull the forecast down. The ingest step will need an explicit check for missing days before this rule is safe on real data.

4.3 LightGBM instead of a seasonal average, and instead of deep learning

Chose: LightGBM, a gradient-boosted tree model. **Alternatives:** a recency-weighted average per weekday and hour (what the client first described); a neural network. **Why not the average:** it cannot see trend, holidays, or the interaction between them. It was kept as a baseline and LightGBM beat it, but not by a landslide: 9.28 versus 10.34 calls per day of error. **Why not deep learning:** 43,824 rows is

small, there are 14 features, and the model has to be explainable to a client whose brand is trust. Trees on calendar features can be read.

What it costs

Trees cannot extrapolate. The "days since start" feature lets the model separate 2021 from 2025, but for a date beyond the training range the model returns the value for the last leaf it knows, roughly late-2025 behaviour. If volume keeps growing at the seed data's pace, every forecast for 2026 and beyond is a little low, and the further out, the lower. A retrain after each new month of data is the mitigation, and it is a manual step.

4.4 Poisson objective instead of ordinary regression

Chose: LightGBM's `objective="poisson"`. **Alternative:** the default squared-error objective, which treats the target like a temperature reading. **Why:** the target is a count of events that arrive independently at some rate. A Poisson objective says two things a plain regression does not: the answer can never be negative, and being off by 2 calls in an hour that usually has 1 is a bigger mistake than being off by 2 in an hour that usually has 10. Think of a bus stop: if a bus comes every 10 minutes on average, the number that show up in an hour is not always 6, but the spread around 6 is predictable and is roughly the square root of 6. Poisson builds that in.

Measured on the seed grid: mean 2.131 calls per hour, variance 3.150. A pure Poisson process would have variance equal to the mean. The ratio is 1.478, which says the raw counts are somewhat more spread out than Poisson. After the model accounts for hour, weekday, and the rest, the remaining spread in the January 2024 test month is 1.168 times what Poisson expects. Mild. The point forecast is unaffected by this; only confidence intervals would be, and the tool does not show intervals yet.

What it costs

Nothing in the point forecast. If intervals are added later, plain Poisson intervals will be about 8% too narrow on this data. A negative-binomial or Tweedie objective would fix that, at the price of one more parameter to explain.

4.5 Calendar features only, no lag features

Chose: 14 features, all computable from the date and the holiday table alone. **Alternative:** add "calls in the same hour last week" or "average of the last 7 days" as inputs, which usually helps short-range forecasts. **Why:** the tool forecasts up to 30 days ahead so a schedule can be built a month out. On day 30 of the forecast, "last week" is itself a forecast, not a fact. Any lag feature would have to be fed its own predictions, and errors would compound day by day.

What it costs

The model cannot react to what happened yesterday. A news event that doubles volume this week will not change next week's forecast until a retrain. The seasonal-naive baseline, which does use the last 8 weeks, is the closest thing the tool has to a reactive forecast, and it is only reported in the backtest, not on the screen.

4.6 One shared `build_features()`

Chose: training and prediction both call the same function in `db/features.py`, and that file is the only place feature logic may live. **Alternative:** compute features in the training script and again, slightly differently, in the forecast script. **Why:** a model trained on one definition of "week of year" and asked about another silently produces wrong numbers with no error. The data-leakage audit confirmed by test that a 2-day forecast grid produces byte-identical feature rows to the same 2 days sliced from the training frame.

What it costs

Every feature must be computable for a future hour. That is what rules out lag features (4.5). It also means the function is called once on the whole 5-year frame before the backtest split, which is safe only while every feature depends on its own row. Adding a rolling feature there would leak future data into training instantly. A comment in the code says so; a unit test does not yet enforce it.

4.7 Passing the training epoch explicitly

Chose: `build_features()` takes an `epoch` argument; the forecast passes the training start date recorded in `models/metadata.json` (2021-01-01). **The bug this prevents:** the trend feature `days_since_epoch` counts days from the first row of whatever frame it is given. Left alone, a forecast for January 2026 would start counting at 0, the model would read "day 0" as January 2021, and every prediction would be 2021-sized. This was caught in review before it shipped.

What it costs

The argument is optional and defaults to "first row of this frame", which is right for training and wrong for everything else. A future caller who forgets it gets a plausible-looking wrong forecast with no error raised. The audit recommended making the argument required; that is still open.

4.8 Erlang C instead of Erlang A

Chose: Erlang C, the standard call-center staffing formula. **Alternative:** Erlang A, which adds one more input: how long callers are willing to wait before hanging up. **Why:** Erlang C has no free parameters beyond the load and the target, is published in every call-center reference, and could be

verified against an outside calculator (it was: the erlang.com worked example of 400 calls per hour, 180-second calls, 30-second average delay, gives 23 agents in both). It was the right first step.

The assumption that is false for us

Erlang C assumes every caller waits forever. Ours do not. In the seed data 8.06% of calls were abandoned, and the median abandoning caller gave up after 69 seconds. The column that measures this, `abandon_sec`, is in the database and is not read by any code. When callers hang up, the queue is shorter than Erlang C thinks, so Erlang C over-staffs slightly during normal hours. But it also hides the cost of under-staffing: with Erlang C, a bad hour shows up as long waits; in reality it shows up as veterans hanging up on a crisis line. Erlang A would report abandonment as its own number. This is the most important open item.

4.9 Rolling-origin backtest instead of a random split

Chose: 24 folds. For each month from January 2024 to December 2025, train on every hour before that month, predict the whole month, score it. **Alternative:** shuffle all hours, hold out 20% at random.

Why: a random split lets the model train on Tuesday 14:00 and be tested on Tuesday 15:00 of the same week. That is not forecasting. The rolling design is the only one that matches how the tool is used: predict a month you have not seen.

What it costs

Time. The full backtest fits the model 24 times plus once more on everything (55 seconds on the development laptop). And every reported metric is an average over 2024 and 2025 only, so it says nothing about how the model would have done in 2021 or 2022.

4.10 Staffing a shift at its peak hour, not its average

Chose: a shift's staff number is the largest hourly requirement inside it. **Alternative:** the average or the total divided by hours. **Why:** people work whole shifts. If the Evening shift needs 4 people at 17:00 and 5 people at 21:00, scheduling 4.5 is meaningless and scheduling 4 misses the target every evening at 21:00. You staff for the peak or you miss the service level at the peak, and the client's public promise is answering inside 30 seconds.

What it costs

Overstaffing in the quiet hours of every shift. On the sample Monday in Section 6, Evening needs 5 from 18:00 onward but only 4 at 16:00 and 17:00. Those two hours carry one idle person. Split shifts or staggered starts would remove that; the tool does not model them.

4.11 Surfacing the cap of 7 instead of clamping to it

Chose: if the math says 8, show 8, in red, with the word "over cap". **Alternative:** show 7 and move on.

Why: a tool that clamps hides the exact information a manager needs: that the target is unreachable with the people available, and by how much. The cap is a fact about hiring; the requirement is a fact about demand. They belong on the screen together.

What it costs

The screen can show numbers the client cannot act on. With the seed data and default sliders this never happens across the 30-day window; it does at 95% service level (Section 7). Some managers will read a red 8 as a criticism rather than a forecast.

4.12 Seeded synthetic data instead of the client's real data

Chose: build and test the whole pipeline on 93,407 generated calls shaped to the client's verbal description. **Alternative:** wait for the export. **Why:** data agreements at a university health system can take weeks. The pipeline, the tests, the API, and the screen do not care where the rows came from, so all of that could be built and reviewed now.

Exactly what this invalidates

Every accuracy number in this document. The generator planted a 4.5% yearly growth, a 14% step on 2023-07-01, a 20:00 to 22:00 peak, Monday-high weekdays, and fixed holiday multipliers. The model recovered a shape it was, in effect, told to expect. Real calls will have news-driven spikes, campaign launches, referral partnerships going live, and months where nothing looks like last year. The "9.28 calls per day" figure is a statement about the pipeline's plumbing, not about Vets4Warriors. The handle-time numbers (mean 703 seconds) are also seeded; the client never stated a handle time and asked the team to measure it. Until real data lands, the right phrase for the model's real-world accuracy is: **not yet measured**.

5. How the model was trained

Everything in this section is what `db/train.py` does, in order, with the numbers it recorded in `models/metadata.json`.

5.1 Rows, target, features

Rows: 43,824, one per hour from 2021-01-01 00:00 to 2025-12-31 23:00, with zeros filled in for the 7,952 hours that had no calls. **Target:** offered, the number of calls that arrived in the hour. **Features:** the 14 below.

Feature	Meaning
dow	Weekday, 1 = Monday to 7 = Sunday
hour	Hour of day, 0 to 23
month	1 to 12
week_of_year	ISO week, 1 to 53
days_since_epoch	Whole days since 2021-01-01; the trend feature
hour_sin, hour_cos	Hour placed on a circle so 23:00 and 00:00 are neighbours
dow_sin, dow_cos	Weekday placed on a circle so Sunday and Monday are neighbours
is_weekend	1 on Saturday and Sunday
is_holiday	1 if the date is in the holidays table
days_to_nearest_holiday	Signed days to the closest holiday, positive when it is ahead, clipped to plus or minus 14
holiday_volume_multiplier, holiday_evening_multiplier	The holiday table's two numbers, 0 on non-holidays

Model settings: objective Poisson, learning rate 0.05, 31 leaves per tree, at least 20 rows per leaf, 90% of features sampled per tree, seed 42, 500 trees. These were fixed before the backtest and never tuned. The rule was: if the fixed model does not beat the seasonal baseline, stop and report, do not search for settings that happen to win.

5.2 How the backtest folds work

A fold is one rehearsal of real use. Fold 1: pretend it is 2024-01-01. Train on every hour before it (26,280 hours, ending 2023-12-31 23:00). Predict all 744 hours of January 2024. Compare to what actually happened. Then move to 2024-02-01 and repeat with one more month of training data. The last fold trains through 2025-11-30 and predicts December 2025. That is 24 folds. The model never sees an hour from the month it is predicting, and the checks in the leakage audit confirmed the

boundaries by asserting that the last training hour is 23:00 of the day before the first test hour, in every fold examined.

After the backtest, the model is trained one more time on all 43,824 hours and that final version is what the screen uses. Its own accuracy on unseen data is by definition not measured; the backtest numbers are the estimate.

5.3 The two baselines

Seasonal naive: for each test hour, take the same weekday and hour over the 8 weeks before the test month and use the median. Example for Tuesday 2024-01-02 at 21:00: the eight preceding Tuesdays at 21:00 had 7, 4, 4, 2, 4, 5, 1, 6 calls; the median is 4. This is close to what a supervisor would do with a spreadsheet, and it is a strong baseline on data with a steady weekly rhythm.

Overall mean: the average calls per hour over the whole training window, used for every hour. Deliberately dumb. It exists so you can see how much of the model's skill is just "knowing the average" versus knowing the shape.

5.4 What the metrics mean

All four are computed over every test hour of every fold pooled together (17,544 hours), so a fold with more hours counts more. Each is illustrated with the model's real predictions for one day, Tuesday 2024-01-02, from the January 2024 fold.

Hour	Actual	Predicted	Error	Hour	Actual	Predicted	Error
00:00	2	2.304	0.304	12:00	1	3.001	2.001
01:00	2	1.925	0.075	13:00	5	3.048	1.952
02:00	0	1.231	1.231	14:00	4	3.350	0.650
03:00	1	1.159	0.159	15:00	4	3.271	0.729
04:00	2	1.061	0.939	16:00	0	3.463	3.463
05:00	1	1.030	0.030	17:00	3	3.500	0.500
06:00	1	1.257	0.257	18:00	2	3.506	1.506
07:00	1	1.262	0.262	19:00	5	3.786	1.214
08:00	1	2.068	1.068	20:00	2	4.724	2.724
09:00	0	2.610	2.610	21:00	1	4.511	3.511
10:00	2	2.683	0.683	22:00	3	4.078	1.078
11:00	3	2.895	0.105	23:00	2	3.182	1.182
Day	48	64.90	16.90				

"Error" is the absolute difference, predicted minus actual, sign dropped. This was a quiet Tuesday; the model over-predicted most evening hours.

MAE, mean absolute error. Add up the 24 hourly errors, divide by 24. For this day: 1.176 calls per hour. Read it as "in a typical hour, the forecast is off by about this many calls". It treats a miss of 3 as three times as bad as a miss of 1.

RMSE, root mean squared error. Square each error, average them, take the square root. For this day: 1.557. Squaring makes big misses count more, so RMSE is always at least MAE, and the gap between them tells you how lumpy the errors are. Here the 16:00 and 21:00 misses drive it.

MAPE, mean absolute percentage error. Each error divided by the actual, averaged, times 100, using only hours where the actual was above zero (you cannot divide by zero). For this day: 59.7%. On small counts MAPE is harsh and misleading: predicting 2.3 for an hour with 1 call is a 130% error even though it is off by one call. It is reported because people ask for it; do not steer by it.

Daily-total MAE. Sum the 24 predictions, sum the 24 actuals, take the difference, then average that over all days. For this day: 16.90. This is the number the client cares about, because hourly errors partly cancel across a day and the schedule is built per shift, not per hour.

5.5 The measured results

Model	MAE (calls/hour)	RMSE	MAPE %	Daily-total MAE (calls/day)
LightGBM, Poisson	1.256	1.632	53.0	9.28
Seasonal naive (8-week median)	1.336	1.778	56.4	10.34
Overall mean	1.463	1.946	50.8	12.66

Pooled over 24 test months, January 2024 to December 2025, 17,544 hours. The overall mean "wins" MAPE only because under-predicting is rewarded by that metric on small counts; it is the worst model on every other measure.

Per month, LightGBM had the lowest daily-total error in 16 of 24 folds, seasonal naive in 7, the flat mean in 1. LightGBM's best month was March 2024 (6.83 calls per day) and its worst was December 2024 (12.47). December is where the holiday features matter most and where the seasonal baseline was worst too (15.82).

The first fold, January 2024, is a fair single example: LightGBM 11.19 calls per day, seasonal naive 11.44, flat mean 19.66.

How wrong it typically is, in the client's units

On a typical day the forecast of the day's total is off by about 9 calls, on days that average 51 calls. Per hour it is off by about 1.3 calls, on hours that average 2.1. Because staffing rounds to whole people, that hourly error usually moves the answer by zero or one person. Read every shift number as plus or minus one. This is the seed data's answer. The real data's answer is not yet measured.

6. From predicted calls to a staffing number

This section follows one real hour from the forecast all the way to a shift number. The hour is Monday 2026-01-05 at 21:00, the busiest hour of the seed data's typical day. Every number was produced by running the project's own code.

6.1 What an erlang is

An erlang is one person kept busy for one hour. If 5 calls arrive in an hour and each lasts 12 minutes, that is 60 minutes of talking, which is 1 erlang: on average one person is on the phone the whole hour. Two erlangs means on average two people are talking at any instant. It is the unit of workload, and it is the only thing Erlang C needs to know about demand.

$$\text{load (erlangs)} = \text{calls_per_hour} \times \text{aht_sec} \div 3600$$

calls_per_hour	the model's predicted calls for the hour
aht_sec	average handle time in seconds, for that weekday and hour
3600	seconds in an hour

Worked: for 2026-01-05 21:00 the model predicts 5.0523 calls and the AHT table says 623.34 seconds. Load = $5.0523 \times 623.34 \div 3600 = \mathbf{0.8748 \text{ erlangs}}$. Slightly less than one person continuously busy.

6.2 Where the AHT comes from

For each of the 168 (weekday, hour) cells, take every answered call in that cell from the whole history and average its talk time, but weight recent calls more. A call from one year ago counts half as much as one from today; two years ago, a quarter.

$$\begin{aligned} \text{weight} &= 0.5 ^ {(\text{age_days} \div 365)} \\ \text{aht_cell} &= \Sigma(\text{weight} \times \text{agent_sec}) \div \Sigma(\text{weight}) \end{aligned}$$

age_days	days between the call and the newest call in the table
365	the half-life in days; the default in <code>db/aht.py</code>
agent_sec	that call's talk time

A cell with fewer than 30 answered calls falls back to the global weighted mean. On the seed data no cell needs the fallback; cell values run from 573 to 1,024 seconds, with nights longer than days as the client described.

6.3 What Erlang C computes and what it assumes

Erlang C answers one question: with N people available and a load of A erlangs, what is the probability that an arriving call finds everyone busy and has to wait? Call that C. It assumes calls arrive at random with a steady rate, call lengths vary around a fixed average, everyone waits as long as it takes, and the N people are interchangeable and always available. The waiting-forever assumption is the one that is wrong for us (Section 4.8).

An analogy: a bank with N tellers. Erlang C says how often a customer walks in to find no free teller. Add a teller and that chance drops; add load and it climbs. It climbs steeply as the load approaches N, which is why one more person makes a large difference at small team sizes.

The code does not use the textbook formula with factorials, which overflow past about 170. It walks up a recurrence one server at a time:

$$\begin{aligned} B(0) &= 1 \\ B(k) &= A \times B(k-1) \div (k + A \times B(k-1)) \quad \text{for } k = 1 \dots N \\ C &= N \times B(N) \div (N - A \times (1 - B(N))) \end{aligned}$$

A	load in erlangs
N	number of people on the phones
B(k)	the chance a call is turned away if there were k servers and no queue (Erlang B); a stepping stone
C	the chance an arriving call has to wait at all (Erlang C)

Edge rules in the code: if N is no larger than A the queue grows without bound, so C is 1. If A is 0, C is 0.

6.4 The service-level formula, symbol by symbol

"Waits at all" is not the target. The target is "answered within 30 seconds". Among the callers who wait, the chance of waiting longer than t seconds falls off exponentially with how much spare capacity there is:

$$\text{service_level} = 1 - C \times \exp(- (N - A) \times t \div \text{aht_sec})$$

C	Erlang C from above: share of callers who wait at all
N - A	spare capacity: people minus average people busy
t	the target seconds, 30
aht_sec	average handle time, same as in the load
exp(-x)	e raised to -x; a number between 0 and 1 that shrinks as x grows

Reading it: the share who wait longer than 30 seconds is C times a shrink factor. More spare capacity, shorter calls, or a more generous t all shrink it. Subtract from 1 to get the share answered in time.

6.5 The full worked example

Hour: Monday 2026-01-05 21:00. Predicted calls 5.0523. AHT 623.34 seconds. Load A = 0.8748 erlangs. Target: 80% within 30 seconds. The code starts at $N = \text{ceil}(A) + 1 = 2$ and climbs; the table starts at 1 so you can see why 1 fails.

N	Erlang C	$\exp(-(N-A) \times 30 \div 623.34)$	Service level	Clears 80%?
1	0.8748	0.9940	0.1305	no
2	0.2662	0.9473	0.7478	no
3	0.0652	0.9028	0.9411	yes, stop here
4	0.0130	0.8604	0.9888	(not needed)
5	0.0022	0.8199	0.9982	(not needed)

Check one row by hand. $N = 2$: spare capacity $2 - 0.8748 = 1.1252$. Exponent: $-1.1252 \times 30 \div 623.34 = -0.05415$, and e to that power is 0.9473. Service level $1 - 0.2662 \times 0.9473 = 0.7478$. Below 0.80, so 2 people are not enough. At $N = 3$ it jumps to 0.9411. **Agents required: 3.** Notice the jump from 75% to 94%. With less than one erlang of load, one extra person is a huge step, and there is no way to land exactly on 80%.

6.6 The efficiency division, same example

Three people on the phones is not three people on the schedule. Breaks, lunch, outbound follow-up calls, and paperwork take a share of every shift. The efficiency slider is that share's complement: 0.65 means a scheduled person is free for inbound calls 65% of the time.

```
scheduled_needed = ceil( agents_required ÷ efficiency )
effective_agents  = floor( scheduled_needed × efficiency )
service_level_at_scheduled = service level recomputed with N = effective_agents
```

Worked: $3 \div 0.65 = 4.615$, rounded up to **5 people to schedule**. Those 5 give $\text{floor}(5 \times 0.65) = \text{floor}(3.25) = 3$ on the phones, and the service level at 3 is 0.9411, as in the table. The rounding always lands at or above the requirement; the review proved by exhaustive check over every efficiency setting and every agent count up to 20 that `effective_agents` is never below `agents_required`.

Doing it by hand

Predicted calls × seconds per call ÷ 3600 is the load. Start with N one above the load, compute Erlang C and the service level, and go up until it clears the target. Divide by the efficiency, round up. Take the biggest hour in the shift. That is the entire algorithm.

6.7 The shift roll-up, same example

Now do the same for all 24 hours of 2026-01-05 and group by shift. The Evening shift (16:00 to 00:30) covers these hours:

Hour	Predicted calls	Agents required	Staff to schedule
16:00	3.52	2	4
17:00	3.89	2	4
18:00	4.57	3	5
19:00	4.67	3	5
20:00	4.90	3	5
21:00	5.05	3	5
22:00	4.90	3	5
23:00	3.83	3	5

Predicted calls for the shift is the sum of its eight core hours: 35.34. Staff for the shift is the maximum: **5**. The 00:00 hour of the next day, which the Evening shift also covers until 00:30, would join the maximum if it were in the requested range. The full day:

Shift	Predicted calls	Agents required (max)	Staff to schedule (max)	Over cap of 7?
Overnight 00:00 to 08:30	14.69	2	4	no
Day 08:00 to 16:30	27.76	2	4	no
Evening 16:00 to 00:30	35.34	3	5	no

Why Overnight needs as many people as Day here

Overnight has half the calls of Day (14.69 versus 27.76), yet both need 2 on the phones and 4 on the schedule. Two reasons. Overnight calls are long in the seed data (AHT 813 to 1,008 seconds versus 586 to 663 by day), so each call carries more load. And at loads under one erlang, one person cannot reach 80% in 30 seconds, so the answer is 2 for any hour with meaningful traffic, day or night. The client's own estimate was 4 to 5 overnight; the tool says 4. Whether that holds on real data is not yet measured.

7. What each control on the screen actually does

The screen has three inputs. Each one is sent to the API as a query parameter and lands in exactly one place in the math. Tables below were produced by calling `staff_for_hour()` with the values shown, for the same hour as Section 6: Monday 2026-01-05 21:00, 5.0523 predicted calls, AHT 623.34 seconds, load 0.8748 erlangs.

7.1 Date range

What it changes: which hours get forecast. Nothing else. For each hour in the range the features are rebuilt (weekday, hour, month, trend days since 2021-01-01, holiday flags) and the model is asked. The defaults are today through 29 days ahead, built from the browser's local date. The API refuses ranges over 366 days.

Which way the numbers move: later dates get a slightly larger trend value, but as Section 4.3 explains the trees do not extrapolate, so a date in 2027 returns roughly what late 2025 looked like. Dates that fall on a holiday in the table get the holiday features; if the range runs past the last holiday in the table (2027-12-31 today), the page shows an amber banner saying so.

7.2 Staff efficiency slider, 50% to 85%

What it changes: only the division in Section 6.6. Agents required is untouched. **Which way:** lower efficiency means each scheduled person contributes less, so more must be scheduled. The number moves in steps because of the round-up.

Efficiency	Agents required	agents ÷ efficiency	Staff to schedule	Service level delivered
0.50	3	6.00	6	0.941
0.55	3	5.45	6	0.941
0.60	3	5.00	5	0.941
0.65 (default)	3	4.62	5	0.941
0.70	3	4.29	5	0.941
0.75	3	4.00	4	0.941
0.80	3	3.75	4	0.941
0.85	3	3.53	4	0.941

Target held at 80%. The delivered service level never changes because the people on the phones (3) never change; only how many bodies it takes to get 3 changes.

7.3 Service level target slider, 80% to 95%

What it changes: the threshold in Section 6.5 that the climbing loop must clear. **Which way:** a stricter target can only raise agents required, which then raises the schedule through the division.

Target	Agents required	Staff to schedule at 0.65	Service level delivered
0.80 (default)	3	5	0.941
0.85	3	5	0.941
0.90	3	5	0.941
0.95	4	7	0.989

At 95% this hour hits the cap of 7 exactly; the screen shows it in amber as "at cap".

7.4 Why raising the target from 80% to 90% sometimes adds two staff and sometimes none

Because the service level moves in big jumps at small team sizes, and the schedule moves in big jumps through the round-up. At 21:00 above, 3 people already deliver 94%, so a 90% target changes nothing: zero added. Scanning the other hours of the same Monday with the code:

Hour	Load	Agents at 80%	Agents at 90%	Schedule at 80%	Schedule at 90%	Added
04:00	0.287	2	2	4	4	0
12:00	0.648	2	3	4	5	1
21:00	0.875	3	3	5	5	0
22:00	1.265	3	4	5	7	2

Three things are happening. First, whether one more agent is needed depends on how close the current service level sits to the new target; 12:00 was just under 90% with 2 agents, 21:00 was well over with 3. Second, the round-up amplifies: going from 3 to 4 agents at 0.65 efficiency is 4.62 to 6.15, which rounds to 5 and 7, a jump of two scheduled people for one extra agent. Third, at 22:00 the calls are long (AHT 930 seconds), so the load is higher and the service level is more sensitive to N. Across the 24 hours of that Monday the change is 0 for 12 hours, 1 for 11 hours, and 2 for one hour. The shift roll-up then takes the maximum, so the Evening shift moves from 5 to 7 on the strength of that single 22:00 hour.

The rule of thumb

At this line's size the answer space is "2, 3, or 4 people on the phones". Each step is a 33% to 50% change in capacity. Sliders do not move the number smoothly; they move it when a threshold is crossed, and the round-up can turn one crossed threshold into two scheduled people. This is a property of small call centers, not a bug.

8. Known limitations and what's next

Three independent reviews were run on 2026-09-13: a data-leakage audit of the training script, a math audit of the queueing and staffing code, and a run-from-clean-copy audit of the whole MVP. Their reports are in `reviews/`. Items they raised that were fixed the same day: the wrong test expectation, the holiday table ending in 2025, working-directory-relative paths, no pinned dependencies, the missing note about `Libomp` on macOS, UTC default dates on the page, no caching of the model between slider moves, and tests that failed instead of skipping without a database. What follows is what is still open.

Model and data

- **No real data.** Every number in this document is from seeded calls. Real-world accuracy: not yet measured. First action when the export arrives: run the ingest, then `make train`, and read the new backtest table before believing anything.
- **Erlang A.** Callers hang up (8.06% in the seed) and `abandon_sec` is unused. Erlang A takes a patience distribution from that column and reports expected abandonment. It is the single biggest correctness upgrade.
- **Holiday multipliers.** The two multiplier features are constants of unknown origin, identical every year, and the "evening" one applies to all 24 hours. On the July 2024 fold the model did slightly better without them (11.17 versus 11.92 calls per day). The audit recommends either dropping them or recomputing them from data before 2024 and restricting the evening one to evening hours.
- **Trees do not extrapolate.** Forecasts beyond the training window flatten at late-2025 levels. A retrain per month of new data is the only mitigation, and it is manual.
- **Outbound work.** Not forecast and not subtracted from capacity. The client has a 2,200-person ongoing-support caseload whose follow-up calls consume the same people. The efficiency slider is the only place this can be expressed today.
- **No confidence intervals.** Only point forecasts. Counts are about 1.17 times more spread out than Poisson after modeling, so naive Poisson intervals would be too narrow.

Code

- **Search start over-states hourly agents at very low loads.** The loop starts at $\text{ceil}(\text{load}) + 1$, but $\text{ceil}(\text{load})$ is already stable for non-integer loads. On the seed forecast this adds one agent in 10.2% of hours, all with load 0.21 erlangs or less, between 03:00 and 08:00. Shift-level output is unchanged in all 5,478 shifts checked, because every shift contains a busier hour. The specification said $\text{ceil}(\text{load}) + 1$; the code follows it.
- **The `epoch` argument is optional.** A future caller that omits it gets 2021-sized forecasts with no error. Should be required.
- **No guard against history-dependent features.** `build_features()` is safe only while every feature depends on its own row. A comment says so; no test enforces it.

- **Reproducibility not pinned.** Seed is set, but LightGBM's `deterministic` and `num_threads` are not, so another machine may produce slightly different numbers.
- **Metadata can be misread.** `models/metadata.json` lists the backtest metrics next to a train end of 2025-12-31; the saved model was refit on all data, so those metrics are not its own out-of-sample error.
- **Handle time may be incomplete.** `agent_sec` is talk time; whether after-call work exists and should be added is unknown. The maximum is exactly 7,200, which looks like a cap.
- **Small unreachable edge cases** in the shift roll-up (partial shifts at the edge of a window, duplicate hours) and in the service-level helper (negative result when agents do not exceed load). Neither can be triggered through the API today.
- **The README states the hourly mean as "roughly 1.6".** The grid mean is 2.13. The README needs correcting.

Operations

- The environment file is loaded in five modules and once in the Makefile; one loader would be cleaner.
- `make serve` always runs with auto-reload, which is a development setting.
- No package layout (`__init__.py`), so tests rely on a path hack.
- The holiday table now ends 2027-12-31. Someone has to add 2028 before then.

Questions only the client can answer

- What is the actual average agent time per inbound call, and does it include wrap-up?
- Is the historical figure of roughly 100,000 calls inbound phone only, or all channels?
- When does the 30-second clock start, and is the 80 to 85% figure over all offered calls or answered calls only?
- Does the Verizon/NICE CXone contract already include a workforce-management module?

9. Glossary

Abandoned call

A caller who hung up before an agent answered.

Agents required

People who must be on the phones in a given hour to meet the service-level target, from Erlang C.

AHT (average handle time)

Typical seconds an agent spends on one call; here, talk time on answered calls, recency-weighted, per weekday and hour.

Backtest

Testing a forecast by rerunning history: train on the past, predict a period that was hidden, compare.

Baseline

A deliberately simple forecast the model must beat to justify its existence.

Cap

The client's practical limit of 7 people per shift. Flagged, never enforced by the tool.

Daily-total MAE

Average, over days, of how far the predicted day total was from the actual day total.

Efficiency

The share of a scheduled shift a person is actually free to take inbound calls. Slider, 50% to 85%, default 65%.

Epoch

The fixed start date (2021-01-01) that the trend feature counts days from.

Erlang

One person continuously busy for one hour. The unit of workload.

Erlang A

Erlang C plus a model of callers giving up. Not yet implemented.

Erlang B

The chance a call is turned away when there is no queue; used only as a stepping stone to compute Erlang C.

Erlang C

The chance an arriving call has to wait, given N people and a load in erlangs; assumes nobody hangs up.

Feature

One input column the model reads, such as hour of day or is-holiday.

Fold

One rehearsal in the backtest: one training window and one test month.

Gradient-boosted trees / LightGBM

A model made of hundreds of small decision trees, each trained to fix the errors of the ones before it. LightGBM is the software that builds them.

Hourly grid

One row per clock hour across the whole period, with zero where no calls arrived.

Lag feature

An input built from earlier values of the thing being predicted, such as "calls last week". Deliberately not used.

Leakage

Any way test-period information reaches the model during training or evaluation, making results look better than they would be in real use.

Load

Predicted calls times seconds per call divided by 3,600. Measured in erlangs.

MAE (mean absolute error)

Average size of the forecast miss, sign ignored.

MAPE (mean absolute percentage error)

Average miss as a percentage of the actual, over hours with at least one call. Unreliable on small counts.

Offered

Calls that arrived in an hour, answered or not. The forecast target.

Poisson objective

A training rule that treats the target as a count of independent events, so it cannot go negative and scales its errors to the expected count.

Rolling origin

A backtest where the training window's end moves forward one step at a time and each step predicts the next block.

RMSE (root mean squared error)

Like MAE but big misses count more.

Seasonal naive

Baseline: median of the same weekday and hour over the previous 8 weeks.

Service level

Share of calls answered within a target time. Here, within 30 seconds; slider from 80% to 95%.

Shift roll-up

Taking the largest hourly staff number inside a shift's window as that shift's requirement.

Staff to schedule

Agents required divided by efficiency, rounded up.

Synthetic (seeded) data

Rows produced by a script to resemble the client's description, used until real data arrives.

View

A saved database query that behaves like a table and is recomputed on every read.